

APLIKASI KATALOG MENU DIGITAL CAFE



LAPORAN TUGAS BESAR ALPRO 2

Zhafran Atthaurrahman Alezaryan | 103012500171
Muhammad Keiza Benjamin Andri Purnomo | 103012530078

Dosen Pengampu :
Fauzan Ramadhan, S.Kom., M.Kom.

DAFTAR ISI

1. PENDAHULUAN

- **1.1 Latar Belakang**
- **1.2 Tujuan Aplikasi**

2. DESKRIPSI SISTEM

- **2.1 Gambaran Umum Sistem**
- **2.2 Struktur Data (Struct dan Array)**
- **2.3 Fitur-Fitur Utama Sistem**
- **2.4 Link Github**

3. DOKUMENTASI DAN PENJELASAN KODE

- **3.1 Struktur Data Pendukung**
- **3.2 Algoritma Pengurutan (Sorting)**
- **3.3 Algoritma Pencarian (Searching)**
- **3.4 Fungsi Operasi CRUD dan Diagnosis**
- **3.5 Contoh Running Code**

4. TANTANGAN YANG DIHADAPI

- **4.1 Manajemen Memori dan Kapasitas Statis (Array)**
- **4.2 Validitas Data pada Binary Search**
- **4.3 Penanganan Duplikasi Hasil pada Pencarian Tahun**

5. KESIMPULAN

6. REFERENSI

1. PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi di era digital saat ini telah mengubah lanskap operasional di industri kuliner seperti kafe. Sistem konvensional yang masih mengandalkan pencatatan manual sering kali menghadapi kendala lambatnya pembaruan data, kesulitan mengubah harga, serta keterbatasan pelanggan dalam menyaring menu sesuai preferensi.

Untuk mengatasi hal tersebut, diperlukan sebuah sistem digital berbasis *Command Line Interface* (CLI) yang mampu mengelola katalog menu secara efisien. Dalam ilmu komputer, optimalisasi manipulasi data serta pencarian (*searching*) dan pengurutan (*sorting*) merupakan fondasi penting. Proyek ini mengimplementasikan struktur data array of struct serta algoritma *Sequential Search*, *Binary Search*, *Selection Sort*, dan *Insertion Sort* menggunakan bahasa Go (Golang) untuk membangun sistem manajemen inventaris kafe yang terstruktur dan interaktif.

1.2 Tujuan Aplikasi

- ❑ **Membangun aplikasi berbasis CLI untuk mengelola data katalog menu digital kafe.**
- ❑ **Menerapkan operasi CRUD (*Create, Read, Update, Delete*) untuk memudahkan admin mengelola ketersediaan dan detail menu.**

- Mengimplementasikan algoritma pengurutan (*Selection Sort*, *Insertion Sort*) dan pencarian (*Sequential Search*, *Binary Search*) untuk mempermudah pelanggan mengeksplorasi menu.
- Menyediakan fitur rekapitulasi statistik untuk mengetahui rata-rata harga dan distribusi jumlah menu per kategori.

2. DESKRIPSI SISTEM

2.1 Gambaran Umum Sistem

Sistem ini adalah aplikasi konsol (CLI) yang dirancang untuk memisahkan hak akses antara Admin dan Pelanggan. Admin memiliki wewenang penuh untuk memanipulasi data *back-end* (menambah, mengubah, dan menghapus menu). Sementara itu, Pelanggan memiliki akses *front-end* untuk melihat katalog, mengurutkan daftar harga dari termurah ke termahal, mencari menu spesifik berdasarkan kategori, serta melihat analitik statistik kafe secara *real-time* dari data yang ada di dalam memori saat program berjalan.

2.2 Struktur Data (Struct dan Array)

Aplikasi ini menggunakan struktur data tipe bentukan (struct) dan *Array* statis:

- Menu: Tipe data *struct* yang merepresentasikan satu entitas hidangan. Atribut yang disimpan meliputi ID (string), Nama

(string), Kategori (string), Harga (float64), Komposisi (string), dan Tersedia (boolean).

- **TabMenu:** Tipe data *struct* yang bertindak sebagai kontainer array statis dengan ukuran maksimal yang didefinisikan oleh konstanta `NMAX = 100`. Atribut `n` (integer) digunakan di dalamnya untuk melacak jumlah indeks aktual yang terisi data.

2.3 Fitur-Fitur Utama Sistem

- ☐ **Tambah Menu:** Menambahkan data menu baru ke dalam sistem dengan validasi kapasitas maksimal.
- ☐ **Ubah Menu:** Memodifikasi detail menu yang sudah ada berdasarkan ID.
- ☐ **Hapus Menu:** Menghapus data menu dari sistem berdasarkan ID dengan algoritma pergeseran elemen (*shifting*).

2.4 Link Github: <https://github.com/Zhafran-spec>

3. DOKUMENTASI DAN PENJELASAN KODE

3.1 Struktur Data Pendukung

```
// Struct untuk spesifikasi poin b: rincian harga, komposisi, ketersediaan
type Menu struct {
    ID          string
    Nama        string
    Kategori    string
    Harga       float64
    Komposisi   string
    Tersedia    bool
}

// Struct untuk menampung array of struct
type TabMenu struct {
    data [NMAX]Menu
    n    int
}
```

Penjelasan:

Struct Menu adalah representasi dari salah satu item serta atribut dari item tersebut, Struct TabMenu berfungsi untuk menyimpan data menu kedalam array

3.2 Algoritma Pengurutan (Sorting)

Aplikasi Menu Digital Cafe ini memiliki 2 fungsi Pengurutan

1. Selection Sort (sortSelectionHargaASC): Mengurutkan data berdasarkan angka atau harga satu item secara berurut dari terkecil ke terbesar

2. Insertion Sort (sortInsertionHargaASC): Mengurutkan nilai Harga dari sisa array, lalu menukarnya (swap) ke posisi depan.

3. Insertion Sort (sortInsertionIDDESC): Mengurutkan ID menu dari array dari yang terbesar ke terkecil

3.3 Algoritma Pencarian (Searching)

```

func tampilSatuMenu(m Menu) {
    fmt.Printf("ID: %s | Nama: %s | Kat: %s | Harga: Rp%.2f | Komposisi: %s | Ready: %t\n",
        m.ID, m>Nama, m.Kategori, m.Harga, m.Komposisi, m.Tersedia)
}

func tampilSemuaMenu(T TabMenu) {
    if T.n == 0 {
        fmt.Println("Katalog menu saat ini masih kosong.")
    } else {
        fmt.Println("--- KATALOG MENU ---")
        var i int = 0
        for i < T.n {
            tampilSatuMenu(T.data[i])
            i++
        }
    }
}

// Poin C: Sequential Search berdasarkan Kategori
func searchSequentialKategori(T TabMenu) {
    var katCari string
    fmt.Print("Masukkan kategori (contoh: coffee): ")
    fmt.Scan(&katCari)

    var found bool = false
    var i int = 0
    fmt.Println("--- HASIL PENCARIAN (SEQUENTIAL) ---")
    for i < T.n {
        if T.data[i].Kategori == katCari {

```

Sequential Search (Pencarian Sekuensial):

- **Cara kerja:** Program mencari data dengan cara menelusuri setiap elemen di dalam *array* dari awal (indeks ke-0) hingga akhir (indeks ke T.n).
- **Proses dalam kode:** Fungsi `searchSequentialKategori` akan membandingkan input kategori dari user (`katCari`) dengan `T.data[i].Kategori` satu per satu. Setiap kali menemukan kecocokan, program langsung menampilkannya. Metode ini sangat sederhana dan tidak memerlukan data yang sudah terurut, tetapi akan memakan waktu sedikit lebih lama jika datanya sangat banyak.

```
// Poin C: Binary Search berdasarkan Kategori
func searchBinaryKategori(T *TabMenu) {
    // Binary Search mensyaratkan data harus terurut lebih dulu berdasarkan elemen yang dicari
    sortKategoriASC(T)

    var katCari string
    fmt.Print("Masukkan kategori (contoh: coffee): ")
    fmt.Scan(&katCari)

    var kiri int = 0
    var kanan int = T.n - 1
    var idxFound int = -1
    var done bool = false

    for kiri <= kanan && !done {
        var tengah int = (kiri + kanan) / 2
        if T.data[tengah].Kategori == katCari {
            idxFound = tengah
            done = true
        } else if T.data[tengah].Kategori < katCari {
            kiri = tengah + 1
        } else {
            kanan = tengah - 1
        }
    }
}
```

Binary Search (Pencarian Biner):

- **Cara kerja:** Program membagi ruang pencarian menjadi dua bagian secara terus-menerus. Syarat utamanya: data harus diurutkan terlebih dahulu.
- **Proses dalam kode:** Fungsi `searchBinaryKategori` pertama-tama akan memanggil `sortKategoriASC` untuk mengurutkan menu berdasarkan abjad kategorinya (A-Z). Kemudian, program mencari elemen di posisi tengah (`tengah`). Jika kategori di tengah lebih kecil dari yang dicari, pencarian digeser ke kanan (`kiri = tengah + 1`), begitu pula sebaliknya. Setelah indeks ditemukan, program juga melakukan pengecekan ke belakang/depan untuk memastikan semua menu dengan kategori yang sama (duplikat) ikut tercetak.

3.4 Fungsi Operasi CRUD dan Diagnosis

Operasi CRUD adalah fungsi dasar bagi Admin untuk mengelola *database* (katalog menu) secara dinamis.

- **Create / Tambah Data (tambahMenu):**
 - **Cara kerja:** Berfungsi untuk memasukkan menu baru ke dalam sistem.
 - **Proses dalam kode:** Program mengecek dulu apakah kapasitas memori penuh ($T.n < NMAX$). Jika masih ada ruang, program akan meminta admin mengisi ID, Nama, Kategori, Harga, Komposisi, dan Status. Data ini disimpan pada indeks ke- $T.n$ (elemen kosong pertama di ujung *array*), lalu nilai $T.n$ ditambah 1.
- **Read / Tampil Data (tampilSemuaMenu & tampilSatuMenu):**
 - **Cara kerja:** Membaca dan mencetak rincian data menu agar bisa dilihat oleh Admin maupun Pelanggan.
 - **Proses dalam kode:** Menggunakan perulangan (loop) dari indeks ke-0 hingga $T.n - 1$. Setiap perulangan akan memanggil fungsi tampilSatuMenu untuk mencetak spesifikasi lengkap (ID, nama, rincian harga, komposisi, dan ketersediaan) ke layar menggunakan format `fmt.Printf`.
- **Update / Ubah Data (ubahMenu):**
 - **Cara kerja:** Memperbarui informasi dari menu yang sudah ada (misalnya: update harga atau status stok).
 - **Proses dalam kode:** Admin memasukkan ID menu yang ingin diubah. Program akan melakukan *Sequential Search* untuk mencari ID tersebut. Jika ID cocok ($T.data[i].ID == idCari$), program akan meminta inputan data baru untuk menimpa/mengganti (*overwrite*) data lama pada indeks i tersebut.
- **Delete / Hapus Data (hapusMenu):**
 - **Cara kerja:** Menghapus menu dari katalog.
 - **Proses dalam kode:** Program mencari indeks dari menu berdasarkan ID menggunakan *Sequential Search*. Setelah indeks ditemukan (`idxHapus`), program akan menggeser semua elemen yang ada di sebelah kanan `idxHapus` mundur

satu langkah ke kiri ($T.data[j] = T.data[j+1]$). Langkah terakhir, jumlah elemen ($T.n$) dikurangi 1, sehingga elemen terakhir "terpotong" secara logis dari *array*.

4. TANTANGAN YANG DIHADAPI

4.1 Manajemen Memori dan Kapasitas Statis (Array)

Karena bahasa Go dalam implementasi ini menggunakan *array* statis dengan batas $NMAX = 100$, sistem rentan mengalami *out-of-bounds error* jika data yang dimasukkan melebihi kapasitas memori yang dialokasikan. Hal ini diatasi dengan menambahkan pemeriksaan kondisi $if\ T.n < NMAX$ sebelum mengeksekusi penambahan data, sehingga aplikasi menolak input berlebih dan tetap berjalan stabil tanpa *crash*.

4.2 Validitas Data pada Binary Search

Algoritma *Binary Search* memiliki syarat mutlak, yaitu data harus dalam keadaan terurut sebelum pencarian dilakukan. Tantangan muncul ketika pelanggan ingin mencari kategori tanpa harus repot mengurutkannya terlebih dahulu. Pendekatan yang dilakukan adalah dengan menyisipkan pemanggilan fungsi `sortKategoriASC(T)` secara otomatis di dalam fungsi `searchBinaryKategori(T)` sebelum pencarian dimulai, sehingga validitas data selalu terjamin.

4.3 Penanganan Duplikasi Hasil pada Pencarian Tahun

(Penyesuaian dari template "Pencarian Tahun" menjadi "Kategori" sesuai kode sumber) Dalam konteks menu kafe, satu kategori (contoh: "coffee") pasti dimiliki oleh lebih dari satu item menu (duplikasi kategori). Pada *Binary Search* standar, pencarian akan berhenti ketika menemukan satu indeks yang cocok. Tantangannya adalah bagaimana menampilkan *semua* menu dengan kategori tersebut. Solusi yang diimplementasikan adalah: setelah satu indeks kategori ditemukan, algoritma melakukan pengecekan bergeser ke kiri dan ke kanan secara linear dari titik temu untuk menemukan dan mencetak semua elemen yang memiliki nilai kategori identik.

5. KESIMPULAN

Aplikasi Katalog Menu Digital Cafe (CAFE-MENU) telah berhasil dikembangkan dengan mengaplikasikan konsep-konsep dasar Algoritma dan Pemrograman. Penggunaan struktur data *Array of Struct* terbukti efektif dalam memodelkan basis data statis dalam memori. Pembagian modul antara hak akses Admin dan Pelanggan mempermudah tata kelola aplikasi.

6. REFERENSI

- Modul Algoritma Dan Pemrograman 2(<https://lms.telkomuniversity.ac.id/mod/url/view.php?id=2860245>)
- Hands-On Data Structures and Algorithms with Go([321dao/Hands-On-Data-Structures-and-Algorithms-with-Go](https://github.com/321dao/Hands-On-Data-Structures-and-Algorithms-with-Go))